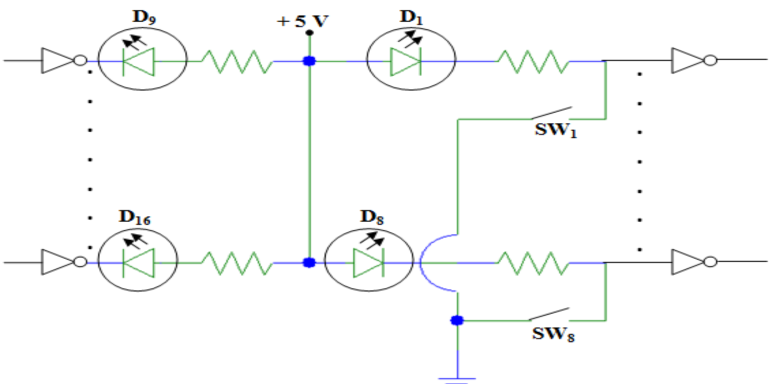
RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

USN	1	R	V							

Department of Electronics & Instrumentation Engineering

Date:05/05/2025	TEST II	Max. Marks:50
Semester: IV	UG	Duration: 90 min
Course Title: Microcontroller & Programming (Common to ECE,E&IE,EEE,E&TE)		Course Code: EI243AI

No	Questions	M	BT	CO
1.	a. Differentiate between the branch and call instruction with program statements. b. Write Thumb-2 assembly program to multiply two 32-bit numbers stored at memory locations 0x20000000 and 0x20000004. Store the result in memory location starting at 0x40000000.	04 06	2 3	1
2.	a. Imagine a **door access control system** where pressing a **push button** at the door triggers a **status LED** to toggle ON or OFF. Design an STM32F407VG-based system to interface a push button and LED to the MCU. Write an application program in C to toggle the LED after a button press event. Use either HAL generated by using STM32CubeMx/register-level programming. Show the interfacing diagram. b. Differentiate between SMLAL and UMLAL with program statements.	06 04	3 2	4 1
3.	a.In a **factory**, workers set operation modes with DIP switches, and the system shows machine status on LEDs. The diagram below shows an interface card with eight DIP switches, with status indicating LEDs D_1 & D_8. The card also features 8 LEDs (D_9-D_{16}) controlled by external devices. Design a suitable scheme to interface the card to the STM32F407VG. Write a program to read the status of switches & display the same on LEDs D_9-D_{16}. Use either HAL generated by using STM32CubeMx/register-level programming.  b. What is the use of the NRST pin of the STM32F407VG MCU? Write a diagram showing the connection of the reset circuit on the NRST pin.	08 02	4 2	4 2
4.	a. Identify the importance of following resets in an MCU: i. Watchdog reset ii. Brown out reset Write a diagram showing the different reset sources in the STM32F407VG MCU. b. Mention different output configurations of a GPIO pin. Explain briefly.	06 04	2	2 3
5.	a. Demonstrate the operations of the following instructions with an example. i. RRX ii. ASR iii. BIC b. Describe the low power modes of STM32F407VG.	06 04	3	1 3

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

USN	1	R	V							

Department of Electronics & Instrumentation Engineering

Date:05/05/2025	**QUIZ II**	**Max. Marks:10**
Semester: IV	**UG**	**Duration: 20 min**
Course Title: Microcontroller & Programming (Common to ECE,E&IE,EEE,E&TE)		**Course Code: EI243AI**

No.	Questions	M	BT	CO
1.	Write the operation of the following assembly instruction: UMULL R2,R3,R0,R1	1	2	1
2.	After performing a data processing operation, the content in register R0 was found to be R0=0x1234F00F. The content in the R0 register was initially R0=0x1234FFFF. Identify the instruction executed.	1	4	1
3.	In the Cortex-M4, the GPIO output values are stored in the _______ register, which holds the current state of the pin (either high or low).	1	1	2

4.	Identify the content of R4 after the execution of the following code segment:
	LDR R1,=0x90002222
	LDR R4,=0x0000
	LSLS R4, R1, #1
		1	3	1

5.	Mention the major difference between JTAG and serial wire debug interfaces.	1	1	1
6.	A conditional execution instruction, ADDNE, appears in the program sequence. Identify the condition & status of the flag for instruction to execute.	1	3	1
7.	Give an example of an interpreted programming language.	1	1	2
8.	What is the advantage of an external pull-up for port pins?	1	2	2
9.	Mention the clock used to drive the RTC of the STM32F407VG MCU.	1	2	1
10.	What is the content of r0 after the execution of the following instruction, and identify the locations where registers are stored? STMDB r0!,{r1-r3}	1	4	1

Department of Electronics & Instrumentation Engineering
Scheme & Solution

<table>
<tr><th>No</th><th>Questions</th><th>M</th></tr>
<tr><td>1.</td><td>
a. Branch Instruction (B)
Transfers program execution to a different location without saving the return address. Used for Loops, conditionals, or jumping unconditionally to another part of the program. No automatic return; cannot return to the point after the branch unless manually programmed.
Example:
B loop_start ; Jump to label 'loop_start', no return address saved
02 Marks
Call Instruction (BL) — Branch with Link
Calls a subroutine/function and saves the return address in the Link Register (LR).Used for: Function calls, subroutine invocation. Return: Use BX LR to return to the instruction following the BL.
Example:
BL my_function ; Branch to 'my_function' and save return address in LR

my_function:
; function code here
BX LR ; Return to the address saved in LR
02 Marks
</td><td>04</td></tr>
<tr><td></td><td>
b.
PRESERVE8
Stack EQU 0x00000100 ;Define stack size

AREA STACK, NOINIT, READWRITE,ALIGN=4
StackMem SPACE Stack

AREA RESET, DATA , READONLY
EXPORT __Vectors ;
__Vectors
DCD StackMem
DCD Reset_Handler

AREA Multiply32bit, CODE, READONLY
ENTRY
EXPORT Reset_Handler
Reset_Handler
Reset_Handler_End

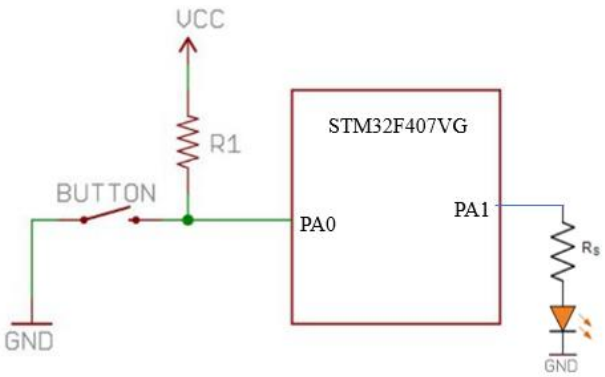
LDR R0, =0x20000000 ; Address of first 32-bit number
LDR R1, [R0] ; Load first 32-bit number into R1

LDR R0, =0x20000004 ; Address of second 32-bit number
LDR R2, [R0] ; Load second 32-bit number into R2

UMULL R4, R5, R1, R2 ; R5:R4 = R1 * R2 (64-bit result)
</td><td>06</td></tr>
</table>

	`LDR R0, =0x40000000 ; Destination address to store result`
	`STR R4, [R0] ; Store lower 32 bits at 0x40000000`
	`STR R5, [R0, #4] ; Store upper 32 bits at 0x40000004`
	`STOP B STOP`
	`END`
	Use of LDR instruction properly:01 Mark
	Use of STR instruction properly:01 Mark
	Definition of Stack, vectors, Reset handler, code segment: 02 Marks
	Use of UMULL instruction properly:01 Mark
	Template of the program: 01 Mark	

2.	a. **Interfacing diagram:** (see below)	06



02 Marks

Program(Assuming HAL is generated using STM32CubeMx):

```
#include "main.h"
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  while (1)
  {

    if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==GPIO_PIN_SET){
          HAL_GPIO_WritePin(GPIOA,GPIO_PIN_1,GPIO_PIN_SET);
          HAL_Delay(500);
          }
 else{
          HAL_GPIO_WritePin(GPIOA,GPIO_PIN_1,GPIO_PIN_RESET);
          HAL_Delay(500);
        }
         }
```

Statement to read switch: 01 Mark

Statements toggle LED: 02 Marks
Template of main code: 01 Mark

b. Differentiate between SMLAL and UMLAL with program statements. 04

b. SMLAL (Signed Multiply Accumulate Long)

- Multiplies two signed 32-bit integers and adds the result to a 64-bit value in destination registers.

Example: SMLAL RdLo, RdHi, Rn, Rm
; RdHi:RdLo = RdHi:RdLo + (signed(Rn) * signed(Rm))

```
LDR R0,=0x1112222(+ve number)
LDR R1,=0x1112222(+ve number)
LDR R3,=0x0000
LDR R2,=0x0000
SMLAL R2,R3,R0,R1
```

After Execution:

R0 * R1= 0x12369 D111 0C84(+ve number)

R2=0x D111 0C84(Lower 32 bits)

R3=0x0001 2369 (Upper 32 bits)

02 Marks

UMLAL (Unsigned Multiply Accumulate Long)

- Multiplies two **unsigned** 32-bit integers and **adds the result** to a 64-bit value in destination registers.

Example: UMLAL RdLo, RdHi, Rn, Rm
; RdHi:RdLo = RdHi:RdLo + (unsigned(Rn) * unsigned(Rm))

```
LDR R0,=0x12345678
LDR R1,=0xABCDEF00
UMULL R2,R3,R0,R1
```

After Execution:

R0 x R1=0xC37 9AAA 42D2 0800

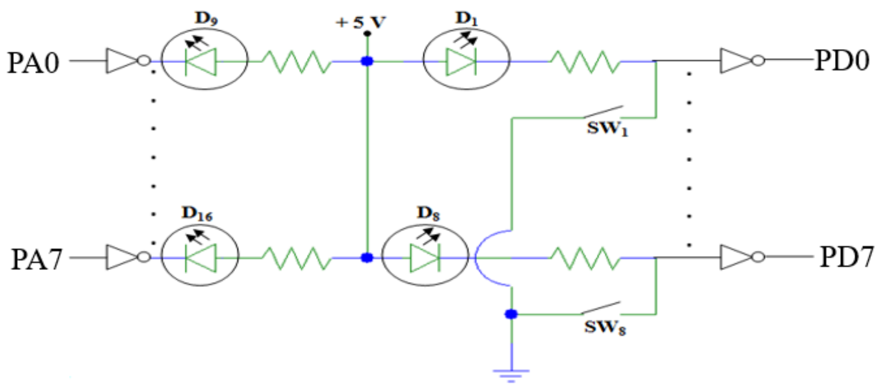
R2=0x42D2 0800(Lower 32 bits)

R3=x0C37 9AAA(Upper 32 bits)

02 Marks

3. a. **Interfacing Diagram:** 08

Department of Electronics & Instrumentation Engineering



02 Marks

Program(Assuming HAL is generated using STM32CubeMx):

```
#include "main.h"
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  while (1)
  {

    if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==GPIO_PIN_SET)
        HAL_GPIO_WritePin(GPIOD,GPIO_PIN_0,GPIO_PIN_SET);
     else
         HAL_GPIO_WritePin(GPIOD,GPIO_PIN_0,GPIO_PIN_RESET);

    if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_1)==GPIO_PIN_SET)
        HAL_GPIO_WritePin(GPIOD,GPIO_PIN_1,GPIO_PIN_SET);
     else
         HAL_GPIO_WritePin(GPIOD,GPIO_PIN_1,GPIO_PIN_RESET);

          .
          .
          .// Similarly, add statements to read other switches and turn on
    corresponding LEDs.
          .
    }
}
```

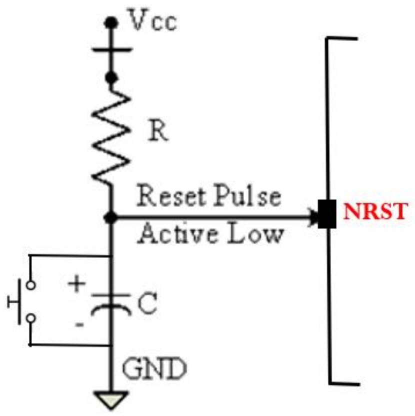
Statement to read switches: 02 Marks
Statements toggle LEDs: 03 Marks
Template of main code: 01 Mark

b. The **NRST pin** on the **STM32F407VG** microcontroller is used for the **hardware reset** of the MCU. 02

Department of Electronics & Instrumentation Engineering

<table>
<tr><td></td><td>

01 Mark



01 Mark

</td><td></td></tr>
<tr><td>4.</td><td>

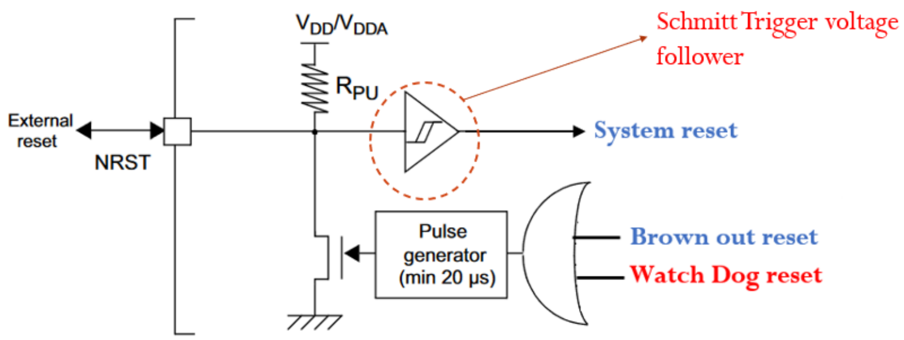
a.
i. **Watchdog Reset:** To automatically reset the MCU if the firmware gets stuck, hangs, or enters an unintended infinite loop. Ensures system reliability by recovering from software faults. Used in safety-critical and autonomous systems (e.g., automotive, medical, industrial) where continuous operation is essential.

02 Marks

ii. **Brown-Out Reset (BOR):** To reset the MCU when the supply voltage drops below a safe operating threshold. Protects the MCU from unpredictable behavior caused by insufficient voltage. Prevents corrupted memory, especially during flash write/erase cycles.

02 Marks

Different reset sources:



02 Marks

b. When the GPIO pin is configured as output, the output buffer is enabled. The port pin can be configured Push push-pull or open-drain, with pull-ups enabled or disabled.

01 Mark

</td><td>06

04</td></tr>
</table>

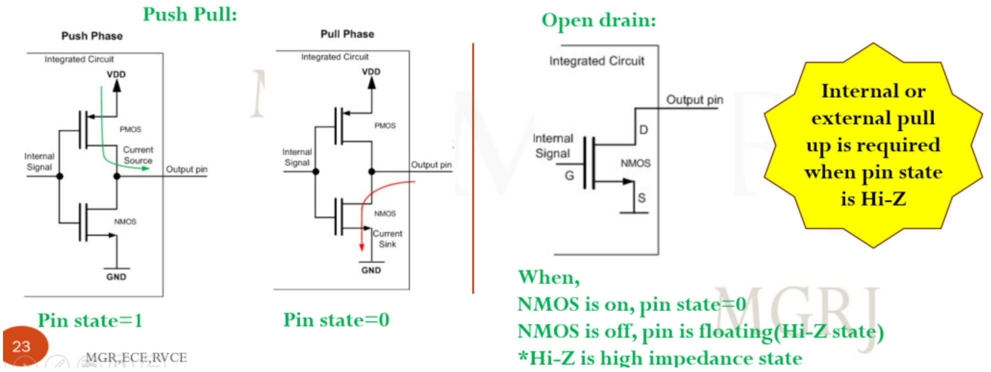


Diagram with brief explanation: 03 Marks

5. a. Demonstrate the operations of the following instructions with an example. 06
i. RRX ii. ASR iii. BIC

a. 04

i. **RRX – Rotate Right with Extend**

- Rotates the bits of a register right by 1 bit, inserting the carry flag (C) into the MSB, and moving the LSB to the carry flag.
  Example: RRX Rd, Rm

  Carry flag=0
  LDR R1,=0x90002222
  RRX R0, R1
  After execution:
  R0=0x48001111

02 Marks

Similarly, explanation of the other two instructions: 04 Marks

b. By default, the microcontroller is in Run mode after a system or a power-on reset. In Run mode, the program code is executed.
Several low-power modes are available to save power when the CPU does not need to be kept running, for example, when waiting for an external event.
The STM32F407xx features three low-power modes:
- Sleep mode (Cortex-M4 core stopped, peripherals kept running)
- Stop mode (all clocks are stopped)
- Standby mode (1.2 V domain powered off)

In addition, the power consumption in Run mode can be reduced by one of the following means:
-Slowing down the system clocks
-Gating the clocks(turning off) to the APBx and AHBx peripherals when they are unused.

02 Marks

Department of Electronics & Instrumentation Engineering

Mode Name	Description	Effect on clocks	Voltage Regulator
Sleep	CPU CLK OFF No effect on other clocks or analog clock sources	No effect	ON
Stop	All 1.2V domain clocks are off	HSE(High Speed External) and HSI(High Speed Internal) clocks are off	ON or in Low power mode
Standby	All 1.2V domain clocks are off		OFF

02 Marks

QUIZ

No.	Questions	M
1.	Performs an unsigned 32-bit × 32-bit multiplication and stores the 64-bit result in two registers.	1
2.	Bitwise OR operation	1
3.	IDR	1
4.	R4=0x20004444	1
5.	JTAG uses 5 wires and SWD uses 2 wires	1
6.	Not equal condition and Z=0	1
7.	JAVA/Python/MATLAB	1
8.	Scourcing current can be controlled	1
9.	LSI RC or LSE Crystal	1
10.	r0=r0-3*4, Mem[R0-12]=r1, Mem[R0-8]=r2, Mem[R0-4]=r3	1